

Diseño y Desarrollo de Videojuegos y Entornos Virtuales

Bomberman HTML5

Ignacio Torres y Nicolas Andrés

Curso 25/26: 3º

29/03/2026

Índice

Introducción	3
Descripción General	3
Descripción Técnica	3
1. Estructura de Archivos	3
2. Sistema de Juego (game.js)	4
3. Sistema de Audio	4
4. Modos de Juego	5
5. Jugadores y Controles	5
6. Sistema de Bombas y Explosiones	5
7. Powerups	6
8. Generación del Mapa	6
9. Modo Online (Firebase)	6
10. Render y Visuales	7
Planificación	7
Diario de Desarrollo	8
Reflexión Personal	8
Obstáculos Principales	8
Enseñanzas Aprendidas	9
Próximos Pasos	9
Conclusión	9
Bibliografía	9

Introducción

Este proyecto tuvo como objetivo recrear el clásico juego Bomberman en un entorno web, utilizando únicamente tecnologías estándar del navegador: HTML5, CSS3 y JavaScript puro, sin necesidad de ningún framework ni motor externo.

Descripción General

El proyecto Bomberman HTML5 es una recreación fiel del videojuego arcade clásico, adaptada para funcionar directamente en el navegador. El juego permite partidas de 2-4 jugadores en local, con la posibilidad de un modo de juego alternativo llamado "Locura", y contemplaba también un modo online a través de Firebase Realtime Database.

El juego se desarrolla sobre un tablero de 15×13 celdas de 48 px cada una, con generación procedural del mapa en cada partida. Los jugadores colocan bombas para destruir bloques rompibles y eliminar a los oponentes, pudiendo recoger powerups que mejoran sus capacidades.

Descripción Técnica

1. Estructura de Archivos

El proyecto está compuesto por los siguientes archivos principales:

- index.html — Estructura HTML del juego, menús e interfaz de usuario.
- style.css — Estilos visuales de la interfaz, overlays y botones.
- game.js — Lógica principal del juego: mapa, jugadores, bombas, explosiones, powerups, audio y bucle de juego.
- online.js — Módulo encargado del modo multijugador online, implementado con Firebase Realtime Database.

2. Sistema de Juego (game.js)

El archivo game.js concentra toda la lógica del juego. Está organizado en secciones claramente comentadas:

- Constantes del grid: CELL (48px), COLS (15), ROWS (13), que definen las dimensiones del tablero.
- Estado global: variables como GS (estado de la partida: menu, playing, paused, gameover), lista de bombas, explosiones, jugadores y powerups.
- Clase Player: gestiona la posición, vidas, velocidad, rango de fuego y lógica de movimiento de cada jugador.
- Bucle de juego (gameLoop): actualiza y renderiza el estado del juego en cada frame usando requestAnimationFrame.

3. Sistema de Audio

El audio del juego está generado de forma procedural mediante la Web Audio API, sin necesidad de archivos externos. Se implementaron los siguientes efectos de sonido:

- place — Sonido al colocar una bomba (onda cuadrada, frecuencia descendente).
- explosion — Ruido blanco con rampa de ganancia descendente para simular una explosión.
- death — Onda de sierra con frecuencia descendente al morir un jugador.
- powerup — Secuencia de tonos ascendentes al recoger un powerup.
- win — Fanfarria al finalizar la partida.

Además, el juego incluye una melodía de fondo (TUNE) compuesta por una secuencia de notas que se reproduce en bucle con osciladores de onda cuadrada. El volumen es ajustable desde el menú de configuración.

4. Modos de Juego

El juego cuenta con los siguientes modos:

- Local (2-4 jugadores) — Los jugadores comparten teclado con controles asignados. Jugador 1: WASD + Espacio; Jugador 2: Flechas + Enter; Jugadores 3 y 4 con teclas IJKL+O y teclado numérico respectivamente.
- Locura — Variante del modo local con parámetros modificados (mayor densidad de bloques, más powerups, mayor velocidad). [Amplía aquí si corresponde.]
- Online (en desarrollo / obstáculo) — Modo multijugador por internet implementado con Firebase. Ver sección 9.

5. Jugadores y Controles

Cada jugador está representado por la clase Player, con los siguientes atributos principales:

- Posición en el grid (col, row) y posición en píxeles para interpolación suave.
- Vidas (3 por defecto), bombas máximas (maxBombs), rango de fuego (fireRange) y multiplicador de velocidad (speedMult).
- Sistema de controles por teclado, con soporte adicional para controles táctiles en dispositivos móviles (initTouchControls).

La detección de colisiones con muros, bloques y explosiones se realiza a nivel de celda del grid.

6. Sistema de Bombas y Explosiones

Al pulsar la tecla asignada, el jugador coloca una bomba en la celda donde se encuentra (placeBomb). Cada bomba tiene:

- Un temporizador de 3 segundos antes de explotar.
- Un rango de explosión (fireRange) que se propaga en las cuatro direcciones cardinales.
- Posibilidad de reacción en cadena si una explosión alcanza otra bomba.

La función explodeBomb calcula las celdas afectadas, destruye bloques rompibles (T.BREAK), puede generar powerups aleatoriamente (probabilidad configurable, por defecto 30%) y mata a los jugadores en el rango de la explosión.

7. Powerups

Al destruir bloques rompibles, existe una probabilidad de que aparezca un powerup. Los tipos disponibles son:

- bomb — Aumenta el número máximo de bombas del jugador (hasta 5).
- fire — Aumenta el rango de explosión.
- speed — Aumenta la velocidad de movimiento del jugador.

Los powerups se recogen automáticamente al pasar por la celda donde están situados.

8. Generación del Mapa

Cada partida genera un mapa procedural mediante la función `generateMap()`:

- Los bordes del tablero son muros indestructibles (T.WALL).
- Las celdas de coordenadas par-par también son muros fijos (patrón clásico de Bomberman).
- Las zonas de spawn de cada jugador (3 celdas adyacentes) se garantizan como suelo libre.
- El resto de celdas se asignan como bloque rompible (T.BREAK) o suelo (T.FLOOR) según la densidad configurada (`blockDensity`, por defecto 0.7).

9. Modo Online (Firebase) — Obstáculo principal

El modo online fue diseñado para permitir partidas entre jugadores en diferentes dispositivos a través de internet, usando Firebase Realtime Database como backend de sincronización.

La arquitectura planteada era la siguiente:

- Un jugador actúa como host y crea una sala con un código único.
- Otros jugadores se unen introduciendo el código de sala.
- El host sincroniza el estado del mapa, bombas y explosiones. Cada cliente envía sus inputs y posición.

Sin embargo, la implementación online no llegó a funcionar correctamente por los siguientes motivos:

- La URL de Firebase Realtime Database configurada en `online.js` apuntaba a la consola de Firebase en lugar de al endpoint REST de la base de datos, lo que impedía realizar peticiones válidas.
- La latencia inherente al modelo de polling (intervalos de sincronización) provocaba desincronización entre el estado del host y los guests, especialmente en eventos rápidos como explosiones en cadena.
- La gestión de listeners con EventSource (SSE) presentó inconsistencias en diferentes navegadores, complicando la recepción de actualizaciones en tiempo real.
- La lógica de autoridad del host (quién decide el estado del mapa) no estaba completamente definida, generando conflictos cuando varios jugadores interactuaban simultáneamente.

Por estas razones, el modo online quedó como funcionalidad no operativa en la versión final entregada. Los modos local y locura funcionan correctamente.

10. Render y Visuales

Todo el renderizado se realiza sobre un elemento `<canvas>` mediante la Canvas 2D API. Las funciones de dibujo principales son:

- `drawFloor / drawWall / drawBreak` — Dibujan cada tipo de celda del mapa con degradados y efectos visuales.
- `drawBombs` — Anima las bombas con efecto de pulsación y mecha.
- `drawExplosions` — Renderiza las explosiones con segmentos (centro, medio, extremo) y efecto de destello.
- `drawPowerups` — Muestra los iconos de cada tipo de powerup en el mapa.
- `drawTitleScreen` — Renderiza la pantalla de inicio animada directamente sobre el canvas.

Planificación

A continuación se detalla la planificación de las tareas realizadas durante el desarrollo del proyecto:

Tarea	Tipo	Persona	Tiempo (h)
[Tarea 1 — ej. Estructura HTML base]	Programación	Ignacio	2
[Tarea 2 — ej. Sistema de mapa procedural]	Programación	Nicolas	3
[Tarea 3 — ej. Clase Player y controles]	Programación	Nicolas	4
[Tarea 4 — ej. Sistema de bombas]	Programación	Nicolas	3
[Tarea 5 — ej. Sistema de explosiones]	Programación	Nicolas	3
[Tarea 6 — ej. Powerups]	Programación	Nicolas	2
[Tarea 7 — ej. Audio procedural]	Programación	Ignacio	2
[Tarea 8 — ej. Modo Locura]	Programación	Nicolas	1.5
[Tarea 9 — ej. Diseño de menús CSS]	Diseño	Ignacio	3
[Tarea 10 — ej. Controles táctiles]	Programación	Ignacio	2
[Tarea 11 — ej. Intentar modo online]	Programación	Nicolas	5
[Tarea 12 — ej. Debug y pulido general]	Programación	Ignacio	3
[Tarea 13 — ej. “Arte” del style.css]	Programación	Ignacio	3
[Tarea 14 — ej. Memoria]	Diseño	Ignacio	5
[Añadir más filas según hacemos]	...	...	...

Diario de Desarrollo

Reflexión Personal

El desarrollo de Bomberman HTML5 ha sido una experiencia enriquecedora y, a la vez, desafiante. Trabajar exclusivamente con tecnologías web nativas —sin motores ni librerías externas— obligó a comprender en profundidad conceptos como el bucle de juego con `requestAnimationFrame`, la detección de colisiones a nivel de grid y la generación procedural de contenido.

Obstáculos Principales

1. Modo Online no funcional:

El obstáculo más significativo del proyecto fue la implementación del modo online. La integración con Firebase Realtime Database resultó más compleja de lo esperado. La URL de la base de datos estaba mal configurada en `online.js` (apuntaba a la consola de Firebase en lugar del endpoint REST), lo que bloqueó completamente la comunicación. Adicionalmente, el modelo de sincronización por polling presentó problemas de latencia que habrían requerido un rediseño arquitectónico para ser viables en producción. Por estos motivos, el modo online no está operativo en la versión final.

2. Sincronización del estado del juego:

Asegurar que el estado del mapa, las bombas y las explosiones fueran consistentes entre el host y los guests online resultó un problema no trivial, especialmente para las reacciones en cadena de bombas que ocurren en fracciones de segundo.

Enseñanzas Aprendidas

- La Web Audio API permite generar efectos de sonido procedurales de calidad sin necesidad de archivos externos, pero requiere una gestión cuidadosa del contexto de audio y los nodos.
- Trabajar con Canvas 2D implica gestionar manualmente todos los aspectos del renderizado, lo que da un control total pero exige más código que soluciones basadas en DOM o WebGL.
- El modelo cliente-servidor para juegos en tiempo real requiere pensar cuidadosamente en la autoridad del estado: quién tiene la versión "verdadera" del juego en cada momento.

Próximos Pasos

- Corregir la configuración de Firebase (endpoint REST correcto) y reimplementar el modo online con una arquitectura más robusta, posiblemente usando WebSockets directos.
- Añadir más tipos de powerups y variedad en la generación de mapas.
- Mejorar los gráficos del canvas con sprites reales en lugar de formas geométricas.

Conclusión

Nuestra participación en Bomberman HTML5 nos ha permitido consolidar conocimientos avanzados de programación web, especialmente en el ámbito de los juegos en el navegador. La implementación de sistemas como el audio procedural con Web Audio API, la generación procedural del mapa y el bucle de juego con Canvas 2D han supuesto un reto técnico significativo que ha ampliado considerablemente mis competencias.

El principal obstáculo del proyecto —el modo online con Firebase— no llegó a estar operativo en la versión final, pero el proceso de intentar implementarlo ha sido en sí mismo una valiosa lección sobre los desafíos del desarrollo de juegos en red, la gestión del estado distribuido y la importancia de una arquitectura bien definida desde el inicio.

Bibliografía

Web Audio API — MDN Web Docs:

https://developer.mozilla.org/es/docs/Web/API/Web_Audio_API

Canvas API — MDN Web Docs: https://developer.mozilla.org/es/docs/Web/API/Canvas_API

Firebase Realtime Database REST API: <https://firebase.google.com/docs/database/rest/start>